Unified Modeling Language (UML)

SENG8323 Topic 3

Unified Modeling Language (UML)

- Unified Modeling Language (UML) is a standard visual modeling language used to specify, visualize, construct, and document the structure and behavior of software systems.
- UML is especially important in object-oriented analysis and design (OOAD) because it provides a common notation for representing classes, objects, processes, interactions, and system architecture.
- UML is a modeling language, not a programming language.
- It helps developers and stakeholders understand a system before and during implementation.

Purpose of UML

UML is used to:

- **Visualize** a software system.
- **Specify** system requirements and design.
- **Document** software architecture and design decisions.
- **Communicate** ideas among developers, analysts, designers, and stakeholders.
- **Model** system structure and behavior before implementation.
- **Identify** problems early in the development process.
- **Support** object-oriented software development.

Main Components of UML

A. Structural diagrams

Structural diagrams describe what the system is made of.

They represent the static structure of the system.

Important structural diagrams include:

1. **Class Diagram**
2. **Object Diagram**
3. **Component Diagram**
4. **Deployment Diagram**
5. **Package Diagram**
6. **Composite Structure Diagram**
7. **Profile Diagram**

B. Behavioral diagrams

Behavioral diagrams describe what the system does.

They represent dynamic aspects of the system.

Important behavioral diagrams include:

1. **Use Case Diagram**
2. **Activity Diagram**
3. **State Machine Diagram**

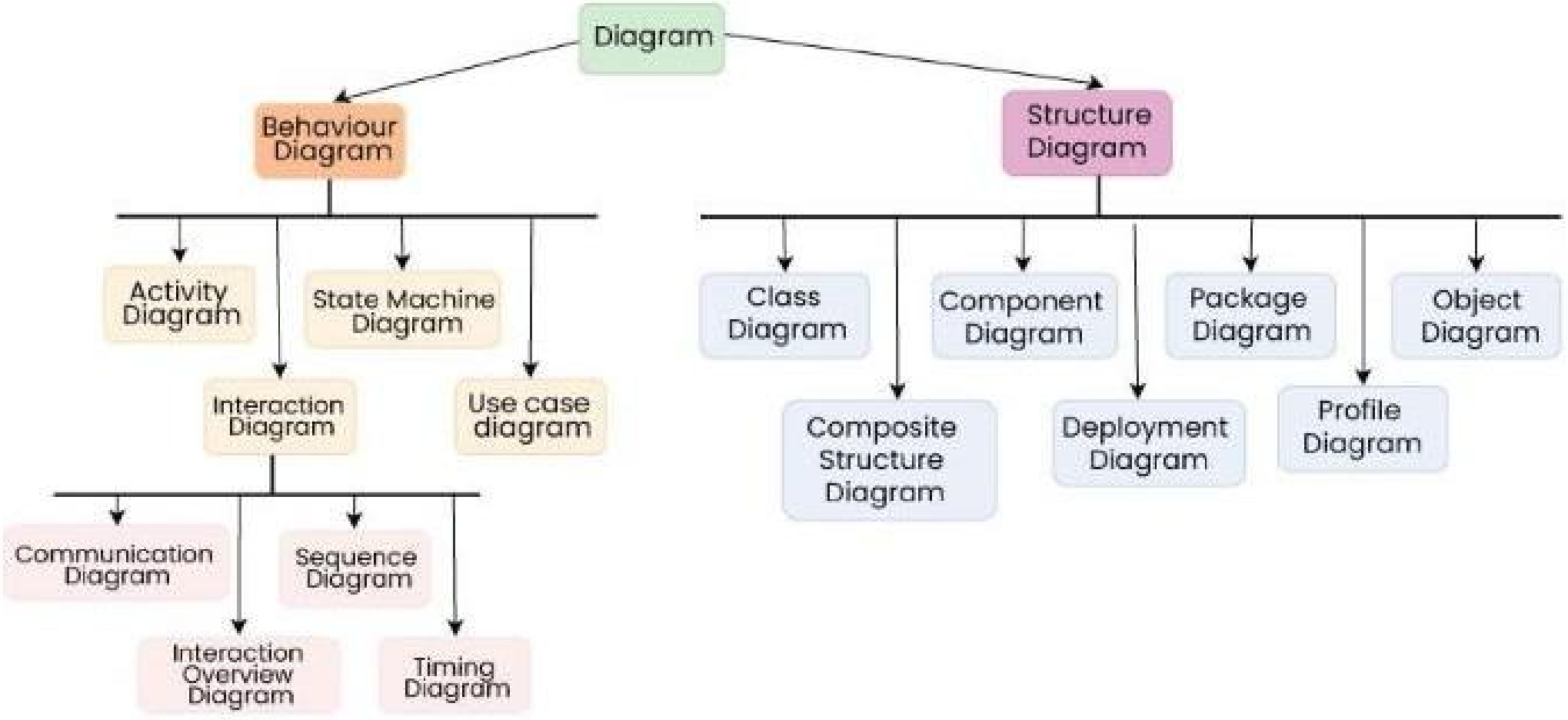
Main Components of UML

C. Interaction diagrams

Interaction diagrams are a subset of behavioral diagrams and focus on how objects communicate with each other. They include:

1. **Sequence Diagram**
2. **Communication Diagram**
3. **Interaction Overview Diagram**
4. **Timing Diagram**

Types of UML Diagrams



Common UML Relationships

Association

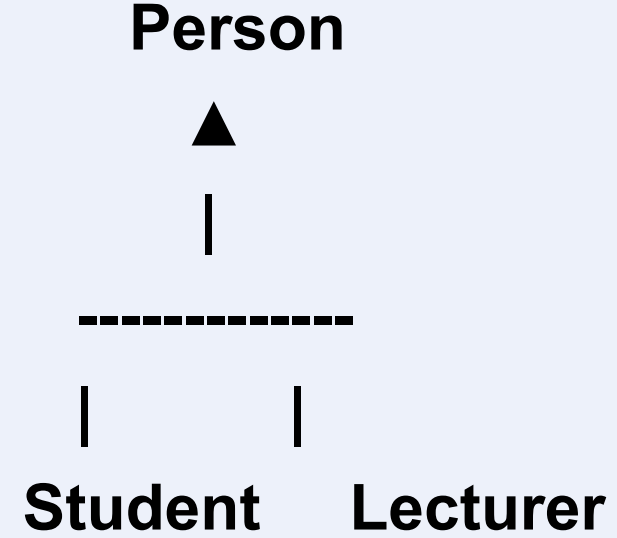
Represents a general relationship between classes.

Student -------- Course

A student enrolls in a course.

Inheritance / Generalization

Represents an "is-a" relationship.



A student is a person.

Common UML Relationships

Aggregation

Represents a weak whole-part relationship.

Department ◇------ Lecturer

The lecturer can exist independently of the department.

Composition

Represents a strong whole-part relationship.

House ◆------ Room

The room is considered a component of the house.

Common UML Relationships

Dependency

Indicates that one element depends on another.

Order - - - - > Payment

The order uses the payment service.

UML Symbols

Symbol	Meaning
Rectangle	Class/component
Stick figure	Actor
Ellipse	Use case
Solid line	Association
Hollow triangle arrow	Generalization/inheritance
Diamond	Aggregation/composition
Dashed arrow	Dependency
Arrow	Direction of relationship/message

UML in the Software Development Life Cycle

UML can be applied throughout the **Software Development Life Cycle (SDLC)**.

SDLC Activity	UML Application
Requirements analysis	Use Case Diagram
System analysis	Class, Activity, State diagrams
System design	Class, Sequence, Component diagrams
Architecture design	Component and Deployment diagrams
Implementation	Class and Component diagrams
Testing	Use Cases, Sequence and Activity diagrams
Documentation	UML diagrams throughout the project

Advantages of UML

1. Easy communication

UML provides a common language for developers, analysts, designers, and stakeholders.

2. Better system understanding

Complex systems can be represented visually.

3. Early error detection

Design problems can be identified before coding.

Advantages of UML

4. Supports object-oriented development

UML directly supports concepts such as:

- Classes
- Objects
- Inheritance
- Encapsulation
- Polymorphism
- Relationships

Advantages of UML

5. Improved documentation

UML diagrams provide useful technical documentation.

6. Supports system maintenance

Future developers can understand the existing system more easily.

7. Encourages standardization

UML provides standardized notation rather than requiring every development team to invent its own diagrams.

UML vs Programming Languages

UML	Programming Language
Modeling language	Implementation language
Used to visualize and design systems	Used to build executable software
Uses diagrams and graphical notation	Uses source code
Platform-independent	May be platform/language dependent
Helps plan the system	Implements the planned system
Example: Class Diagram	Example: Java, C++, Python

UML vs Programming Languages

For example, a UML class:

Student

- name

- studentID

+ register()

+ viewResults()

could be implemented in Java as:

```
class Student {
    private String name;
    private int studentID;

    public void register() {
        // implementation
    }

    public void viewResults() {
        // implementation
    }
}
```

Thus, UML describes the design, while programming languages implement the design.

The most commonly used UML diagrams are:

- **Use Case Diagram** – models system functionality and actors.
- **Class Diagram** – models classes and their relationships.
- **Sequence Diagram** – models object interactions over time.
- **Activity Diagram** – models workflows.
- **State Machine Diagram** – models object states.
- **Component Diagram** – models software components.
- **Deployment Diagram** – models physical deployment.

Use Case Modeling

- **Use Case Modeling** is a software analysis technique used to describe **how users or external systems interact with a software system to achieve specific goals**.
- It focuses on the **functional requirements** of a system from the perspective of its users rather than describing how the system is implemented.
- Use case modeling is an important part of **Object-Oriented Analysis and Design (OOAD)** and is commonly represented using a UML Use Case Diagram.

Purpose of Use Case Modeling

Use case modeling helps to:

- Identify what the system should do.
- Identify the users and external systems that interact with it.
- Capture functional requirements.
- Define the boundaries of a system.
- Improve communication between developers and stakeholders.
- Provide a basis for system design and testing.
- Identify missing or unclear requirements.
- Describe user goals in a simple and understandable way.

Main Elements of Use Case Modeling

Actor

An actor is a person, organization, device, or external system that interacts with the system. Actors are outside the system boundary.

Examples:

- **Customer**
- **Student**
- **Lecturer**
- **Administrator**
- **Bank**
- **Payment Gateway**

An actor is represented in UML by a stick figure.



Customer

Main Elements of Use Case Modeling

Use Case

A use case represents a specific function or goal that the system provides to an actor.

Examples:

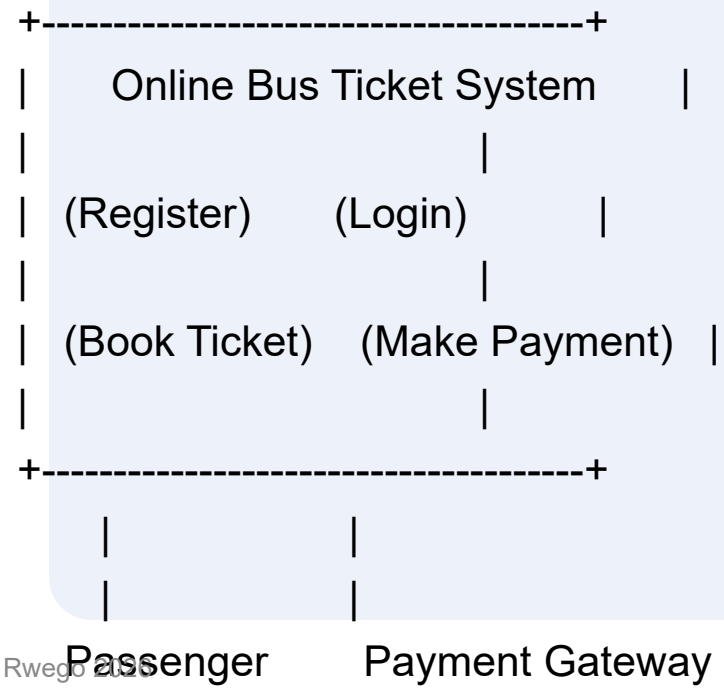
- Register Account
- Login
- Book Ticket
- Make Payment
- Generate Report
- Withdraw Money

A use case is normally represented by an oval.

System Boundary

The system boundary defines what is inside and outside the system.

For example:




Use Case Diagram Notations
Actor
UseCase
System Boundary

Types of Relationships in Use Case Modeling

Association

An association shows that an actor participates in a use case.

Passenger -------- (Book Ticket)

The passenger interacts with the booking functio_{n.}

Include Relationship

An **<<include>>** relationship means that one use case always requires another use case as part of its behavior.

Example:

(**Book Ticket)**

|

| <<include>>

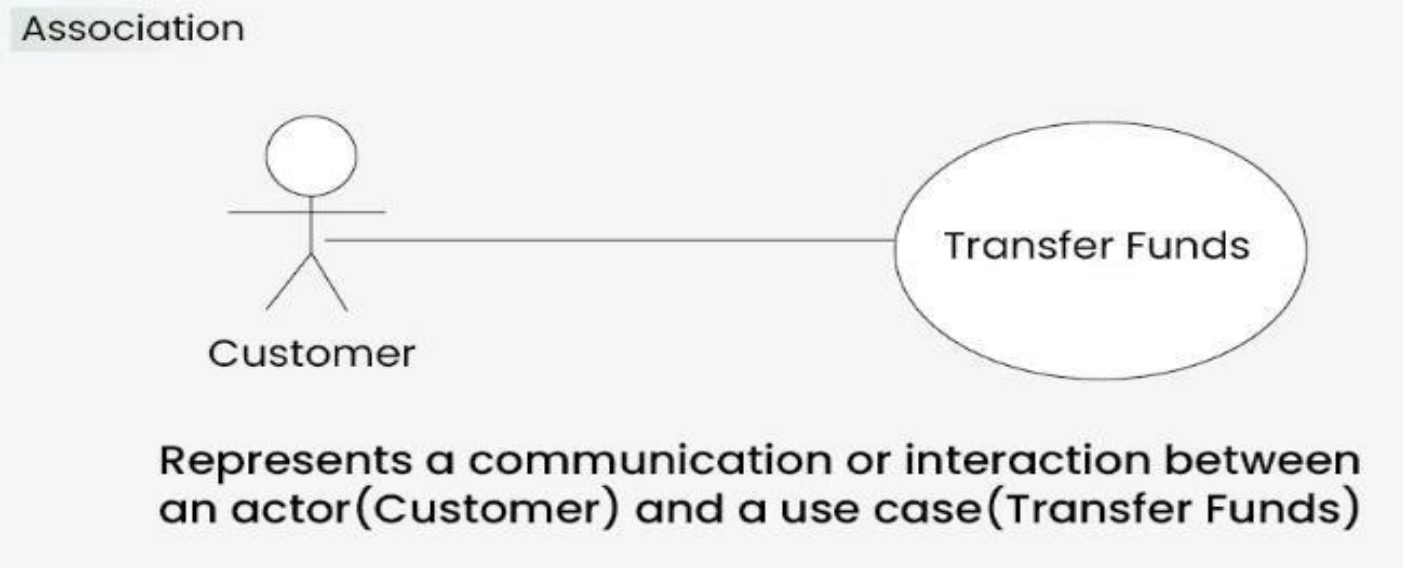
v

(Make Payment)

When a ticket is booked, making payment is always required.

Example: Online Banking System

- **Actor:** Customer
- **Use Case**:Transfer Funds
- **Association:** A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.



Types of Relationships in Use Case Modeling

Another example:

(Book Ticket)

|

| <<include>>

v

(Validate Passenger)

The booking process always validates the passenger.

Key idea

Include = mandatory/reusable behavior.

Extend Relationship

An <<extend>> relationship means that one use case optionally adds behavior to another use case under certain conditions.

Example:

(**Apply Discount)**

|

| <<extend>>

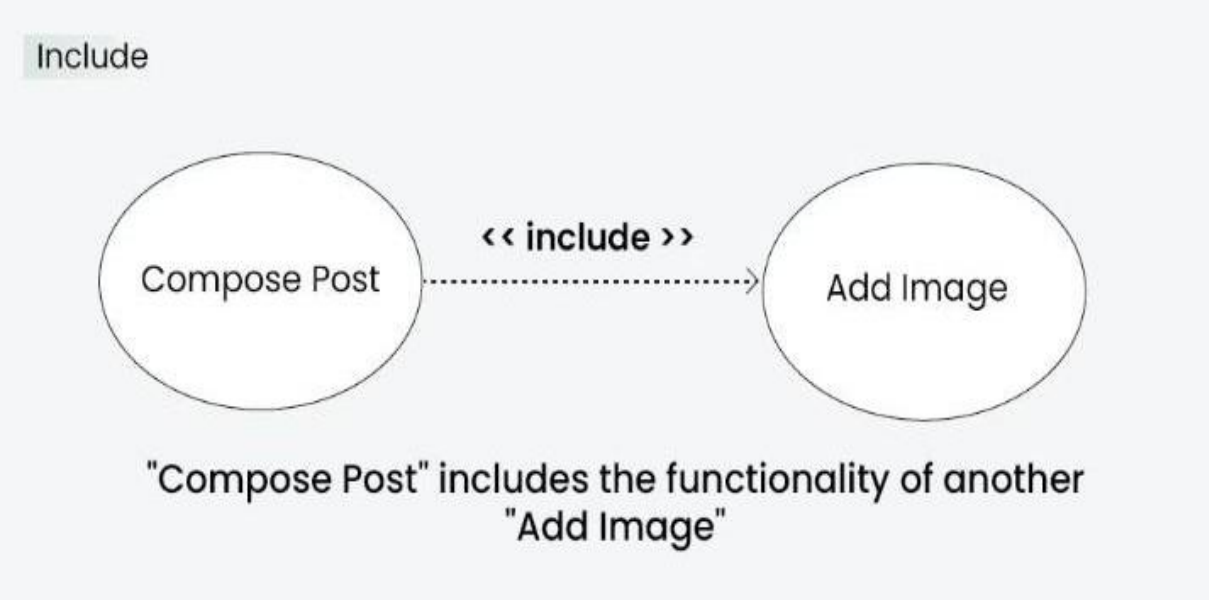
v

(Book Ticket)

A discount may be applied when the passenger has a valid promotional code, but it is not required for every booking.

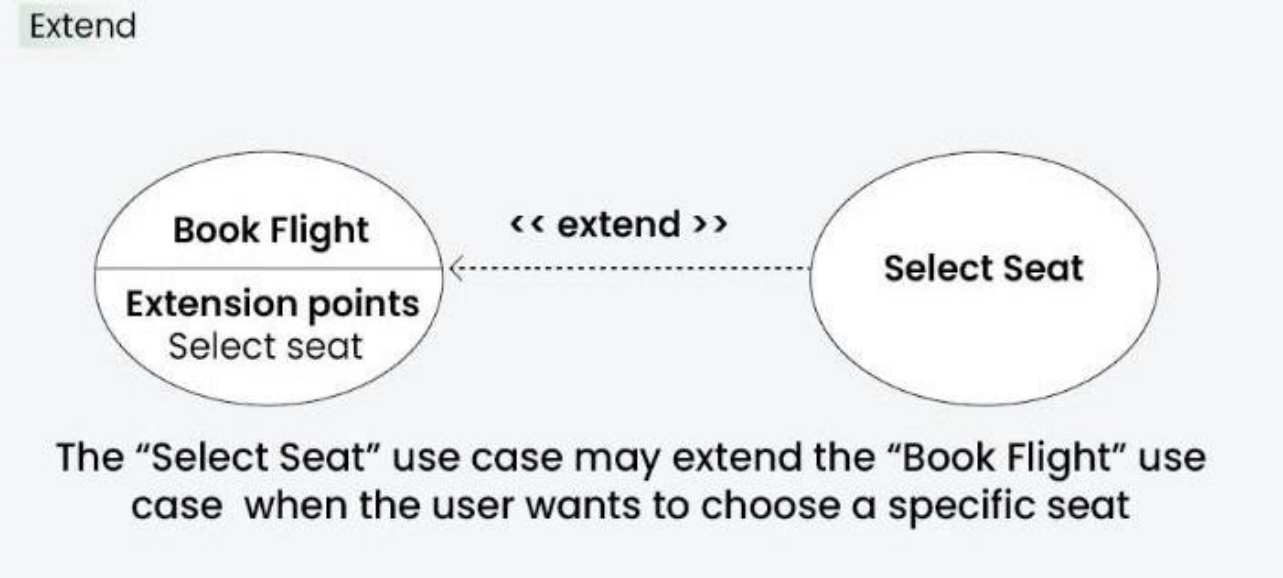
Example: Social Media Posting

- Use Cases: Compose Post, Add Image
- Include Relationship:The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.



Example: Flight Booking System

- **Use Cases:** Book Flight, Select Seat
- **Extend Relationship:** The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.



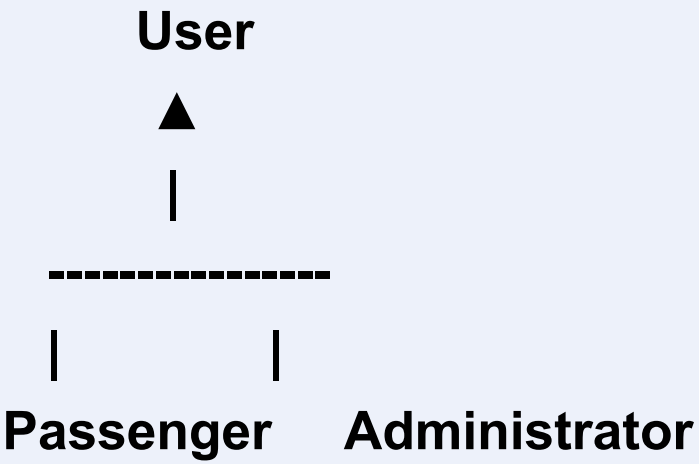
Types of Relationships in Use Case Modeling

Extend = optional or conditional behavior.

Generalization

Generalization represents inheritance between actors or use cases.

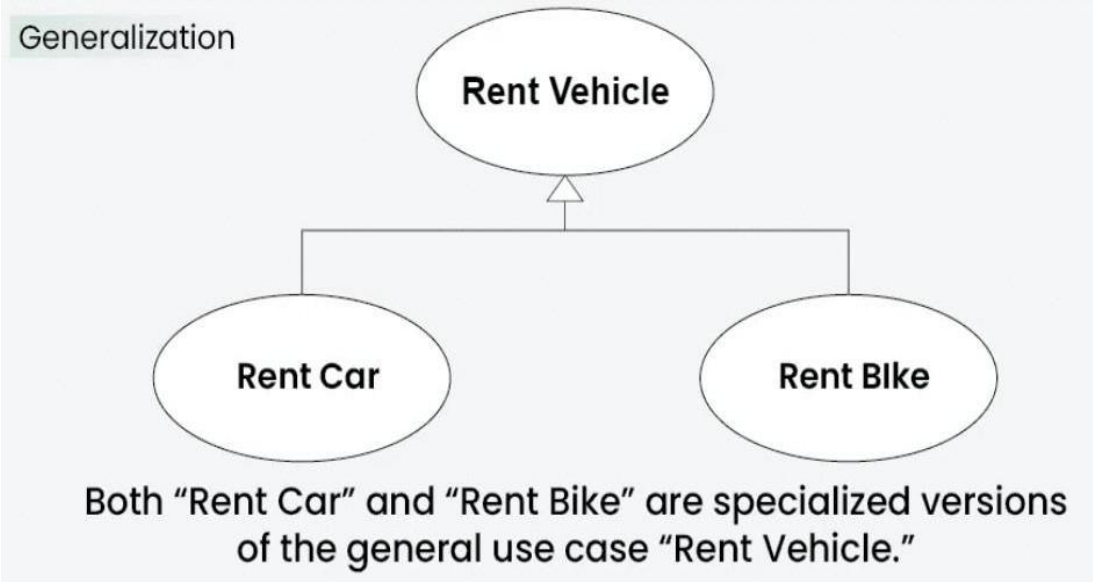
Example:



Both Passenger and Administrator are specialized types of User.

Example: Vehicle Rental System

- **Use Cases:** Rent Car, Rent Bike
- **Generalization Relationship**: Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."



Use Case Diagram example(Online Shopping System)

Actors:

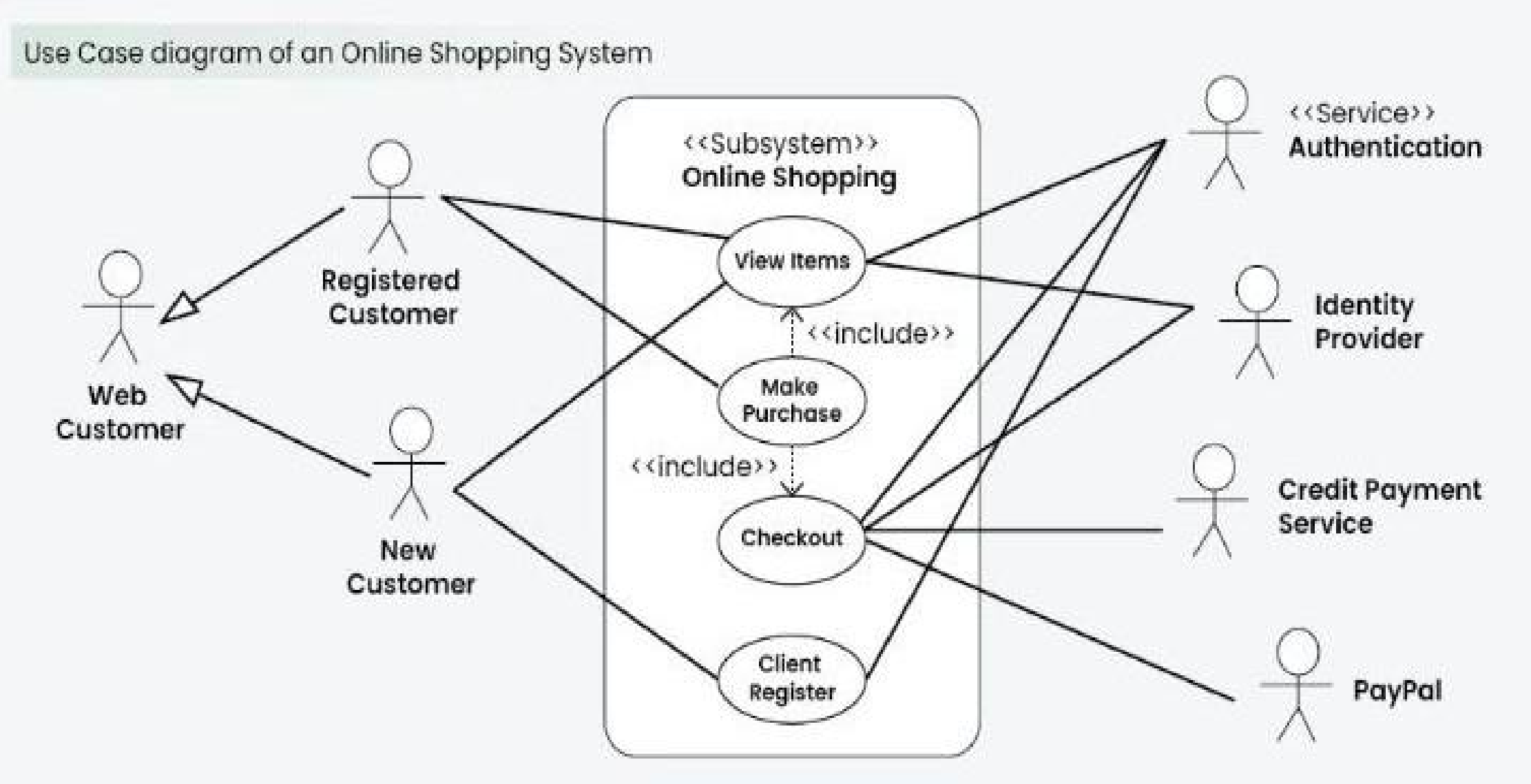
- Customer
- Admin

Use Cases:

- Browse Products
- Add to Cart
- Checkout
- Manage Inventory (Admin)

Relations:

- The Customer can browse products, add to the cart, and complete the checkout.
- The Admin can manage the inventory.

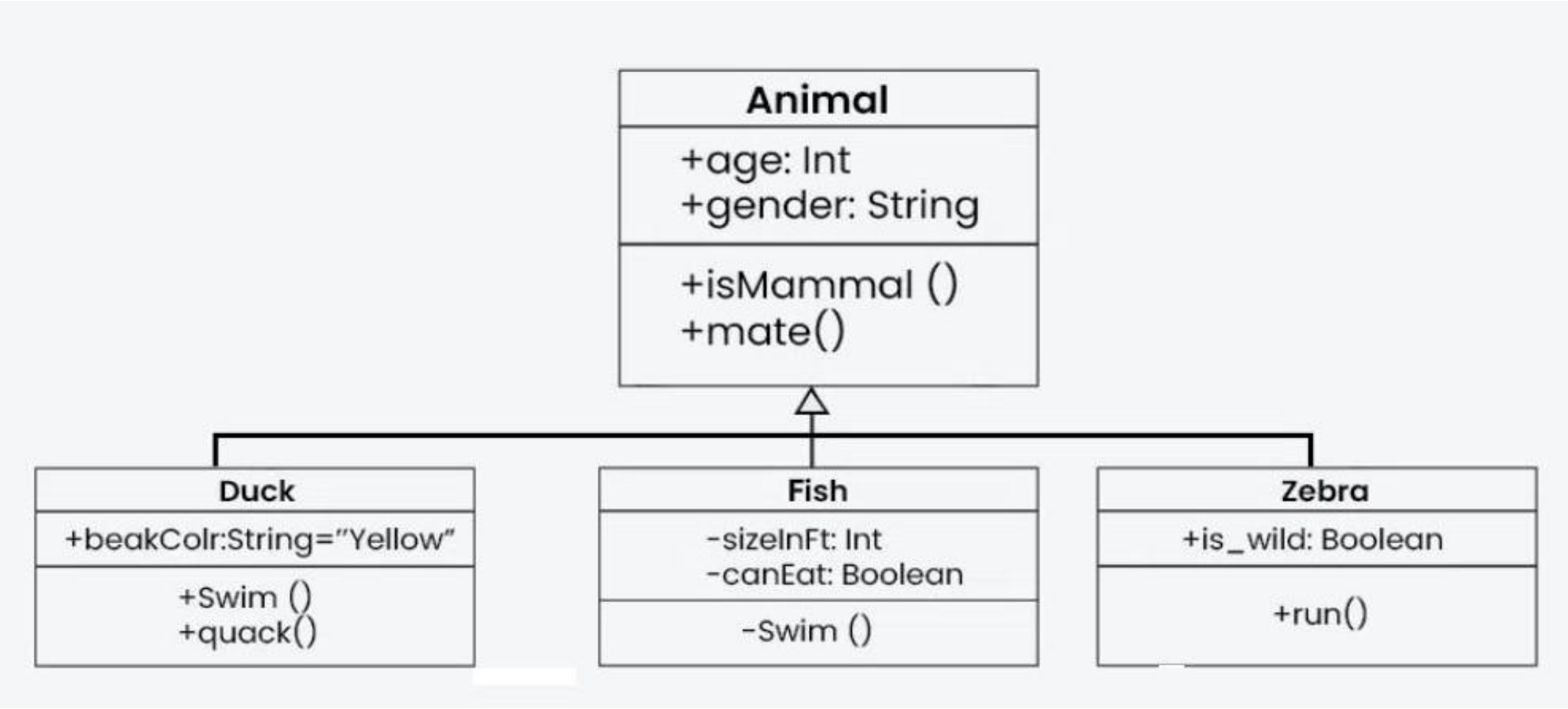

Use Case diagram of an Online Shopping System
<<Subsystem>>
Online Shopping
View Items
<<include>>
Make Purchase
<<include>>
Checkout
Client Register
Registered Customer
Web Customer
New Customer
<<Service>>
Authentication
Identity Provider
Credit Payment Service
PayPal

Class Diagram

- A **Class Diagram** is a UML diagram that represents **the static structure of a software system.** It shows the system's **classes, attributes, methods, and relationships between classes.**
- Class diagrams are one of the most important diagrams in Object-Oriented Analysis and Design (OOAD) because they provide a blueprint for designing object-oriented software.

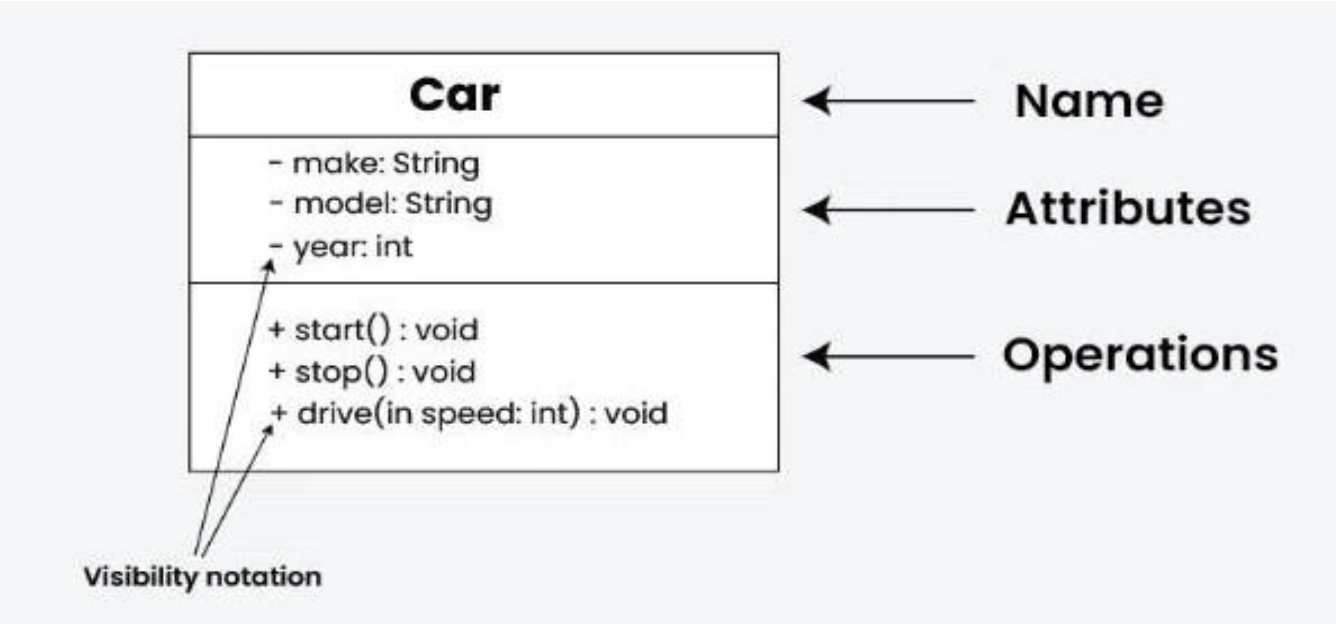
- A UML class diagram visually represents the structure of a system by showing its classes, attributes, methods, and the relationships between them.
- Helps everyone involved in a project—like developers and designers—understand how the system is organized and how its components interact.
- Helps to communicate and document the structure of the software.

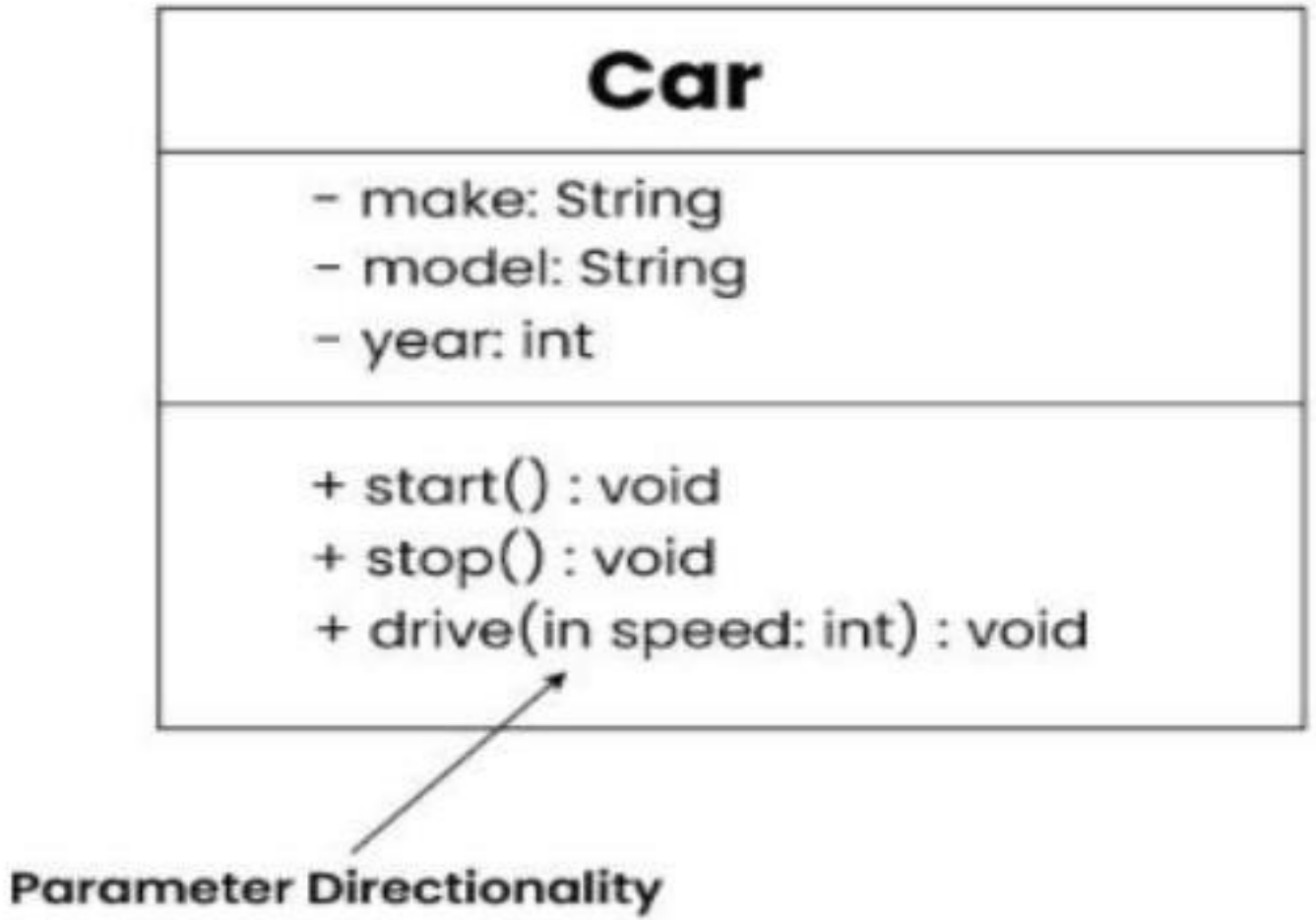
UML Class Diagram example



UML Class notation

- Classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods.




Car
- make: String
- model: String
- year: int
+ start() : void
+ stop() : void
+ drive(in speed: int) : void
Parameter Directionality

1. Association

- An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class. Associations are typically depicted as a solid line connecting the classes, with optional arrows indicating the direction of the relationship.

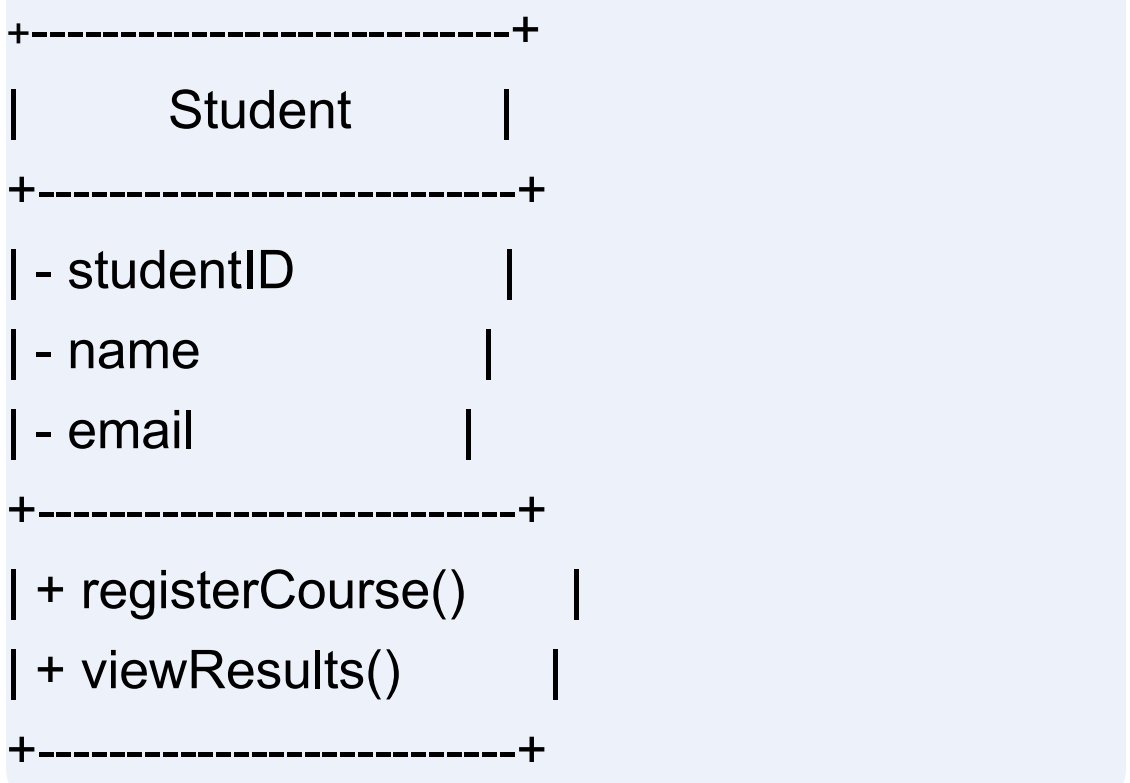
2. Directed Association

- A directed association in a UML class diagram represents a relationship between two classes where the association has a direction, indicating that one class is associated with another in a specific way.

Benefits of Class Diagrams

- Class diagrams represent the system's classes, attributes, methods, and relationships, providing a clear view of its architecture.
- They shows various relationships between classes, such as associations and inheritance, helping stakeholders understand component connectivity.
- Class diagrams serve as a visual tool for communication among team members and stakeholders, bridging gaps between technical and non-technical audiences.
- They guide developers in coding by illustrating the design, ensuring consistency between the design and actual implementation.
- Many development tools allow for code generation from class diagrams, reducing manual errors and saving time.

A simple class is represented as:



A class normally has three sections:

1. **Class name**
2. **Attributes**
3. **Operations/Methods**

Purpose of a Class Diagram

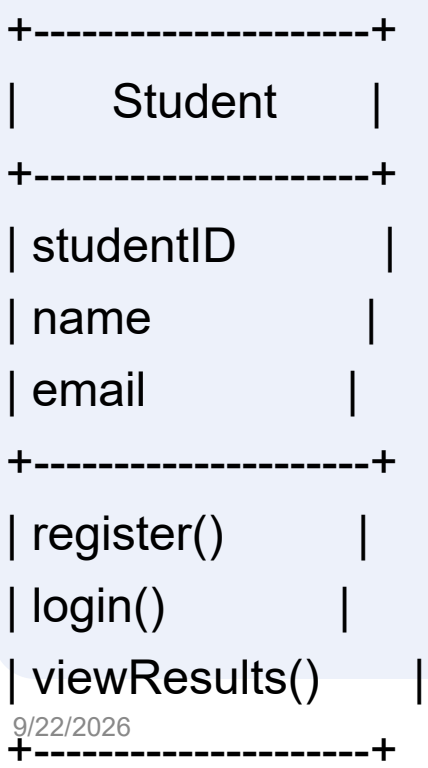
- Model the structure of a software system.
- Identify important classes and objects.
- Define attributes and operations.
- Show relationships between classes.
- Represent inheritance and abstraction.
- Provide a blueprint for implementation.
- Improve communication among developers and stakeholders.
- Support database and software design.
- Help identify responsibilities of different classes.

Main Elements of a Class Diagram

Class

A class is a blueprint that defines the properties and behavior of objects.

Example:



For example, objects created from this class might be:

student1 : Student

student2 : Student

student3 : Student

Main Elements of a Class Diagram

Attributes

Attributes represent the data or properties of a class.

For a Student class:

studentID

name

email

dateOfBirth

programme

Example:

```
+--------------------+
|       Student      |
+--------------------+
| - studentID : int  |
| - name : String    |
| - email : String   |
+--------------------+
```

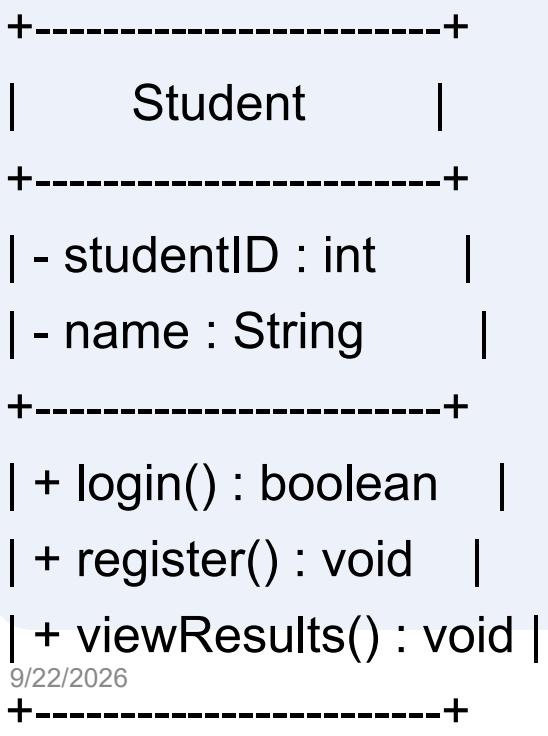
The notation:
attributeName : DataType
is commonly used in UML.

Main Elements of a Class Diagram

Operations/Methods

Operations represent the behavior or actions that a class can perform.

Example:



The notation:

methodName(parameters) : returnType

is commonly used.

Visibility in Class Diagrams

UML uses symbols to indicate the visibility of attributes and operations.

Symbol	Visibility	Meaning
+	Public	Accessible from outside the class
-	Private	Accessible only within the class
#	Protected	Accessible within the class and subclasses
~	Package	Accessible within the same package

Example:

- +-------------------------+
- | Employee |
- +-------------------------+
- | - employeeID : int |
- | - salary : double |
- | # department : String |
- +-------------------------+
- | + calculateSalary() |
- | + displayDetails() |
- +-------------------------+

Class Diagram Development Process

A class diagram can be developed through the following steps:

Step 1: Identify classes

Extract important entities from the requirements.

For an online bus system:

Passenger, Bus, Ticket, Booking, Payment, Route.

- **Step 2: Identify attributes**

Determine the data associated with each class.

For Passenger:

passengerID, name, email, phone.

- **Step 3: Identify operations**

Determine what each class should do.

For Passenger:

login(), bookTicket(), cancelBooking().

- Step 4: Identify relationships

Determine how classes interact.

Examples:

- Passenger → Booking
- Booking → Payment
- Booking → Ticket.

- **Step 5: Define multiplicities**

Specify how many objects participate in each relationship.

- **Step 6: Identify inheritance**

Look for common characteristics that can be placed in a superclass.

- **Step 7: Validate the model**

- Check the class diagram against the system requirements.

Characteristics of a Good Class Diagram

A good class diagram should:

- Represent the important classes of the system.
- Have clear and meaningful class names.
- Clearly define attributes and methods.
- Show appropriate relationships.
- Use correct multiplicities.
- Avoid unnecessary complexity.
- Follow principles of good object-oriented design.
- Be consistent with the system requirements.
- Be easy for developers and stakeholders to understand.

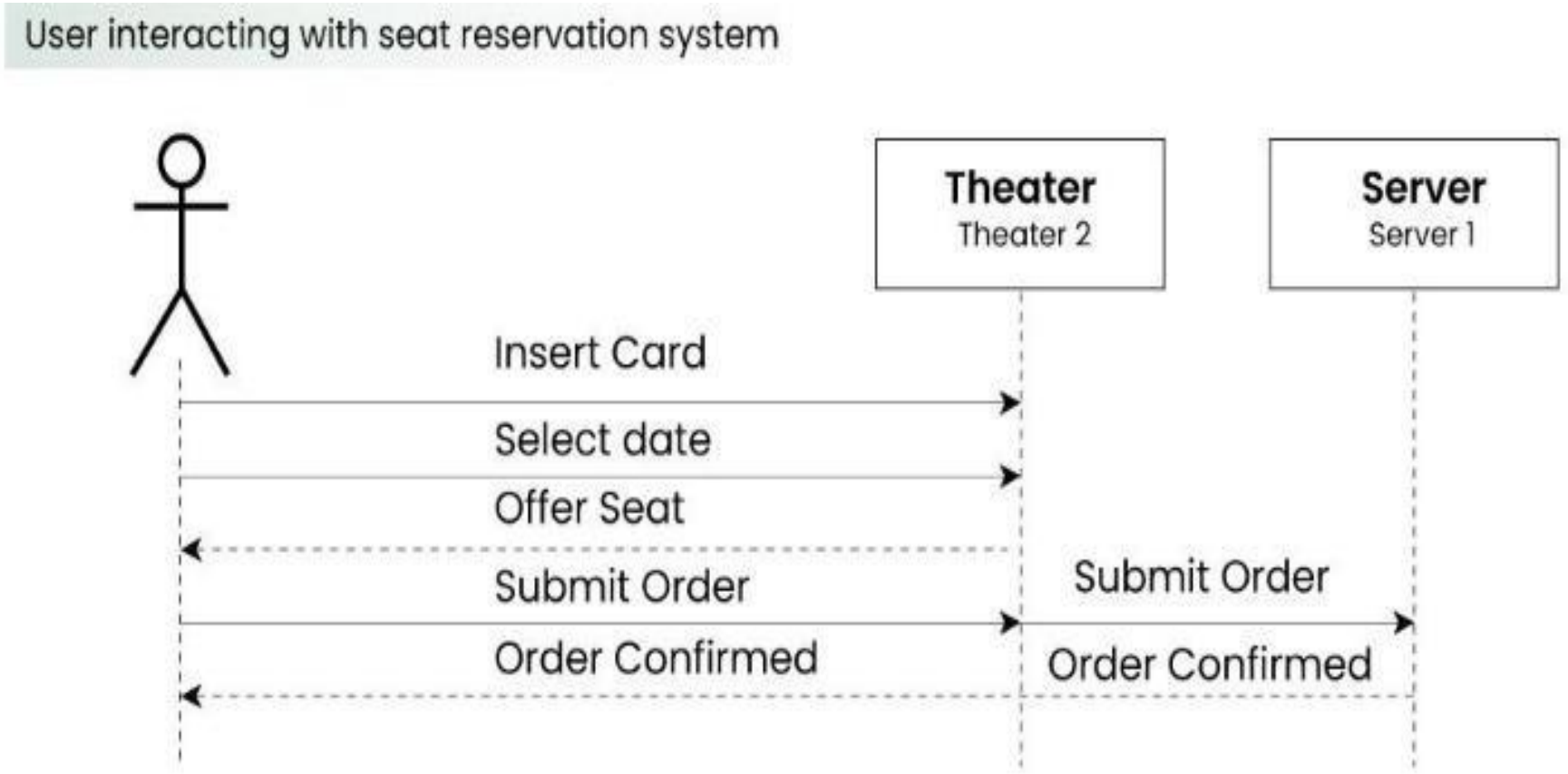
Key Point

- A class diagram is primarily a structural model. It answers questions such as:
- **What classes exist?**
- **What information do they contain?**
- **What operations can they perform?**
- **How are the classes related?**
- In object-oriented software development, the class diagram therefore serves as an important bridge between requirements analysis and program implementation.

Sequence Diagrams - Unified Modeling Language (UML)

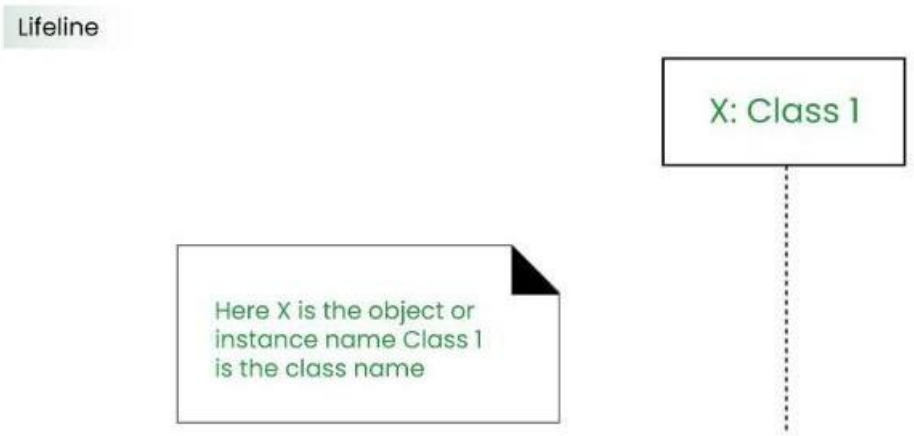
- A Sequence Diagram is a key component of Unified Modeling Language (UML) used to visualize the interaction between objects in a sequential order. It focuses on how objects communicate with each other over time, making it an essential tool for modeling dynamic behavior in a system. Sequence diagrams illustrate object interactions, message flows, and the sequence of operations, making them valuable for understanding use cases, designing system architecture, and documenting complex processes.
- A Sequence Diagram shows how objects communicate in a particular sequence over time.

For example:


User interacting with seat reservation system
Theater
Theater 2
Server
Server 1
Insert Card
Select date
Offer Seat
Submit Order
Submit Order
Order Confirmed
Order Confirmed

2. Lifelines

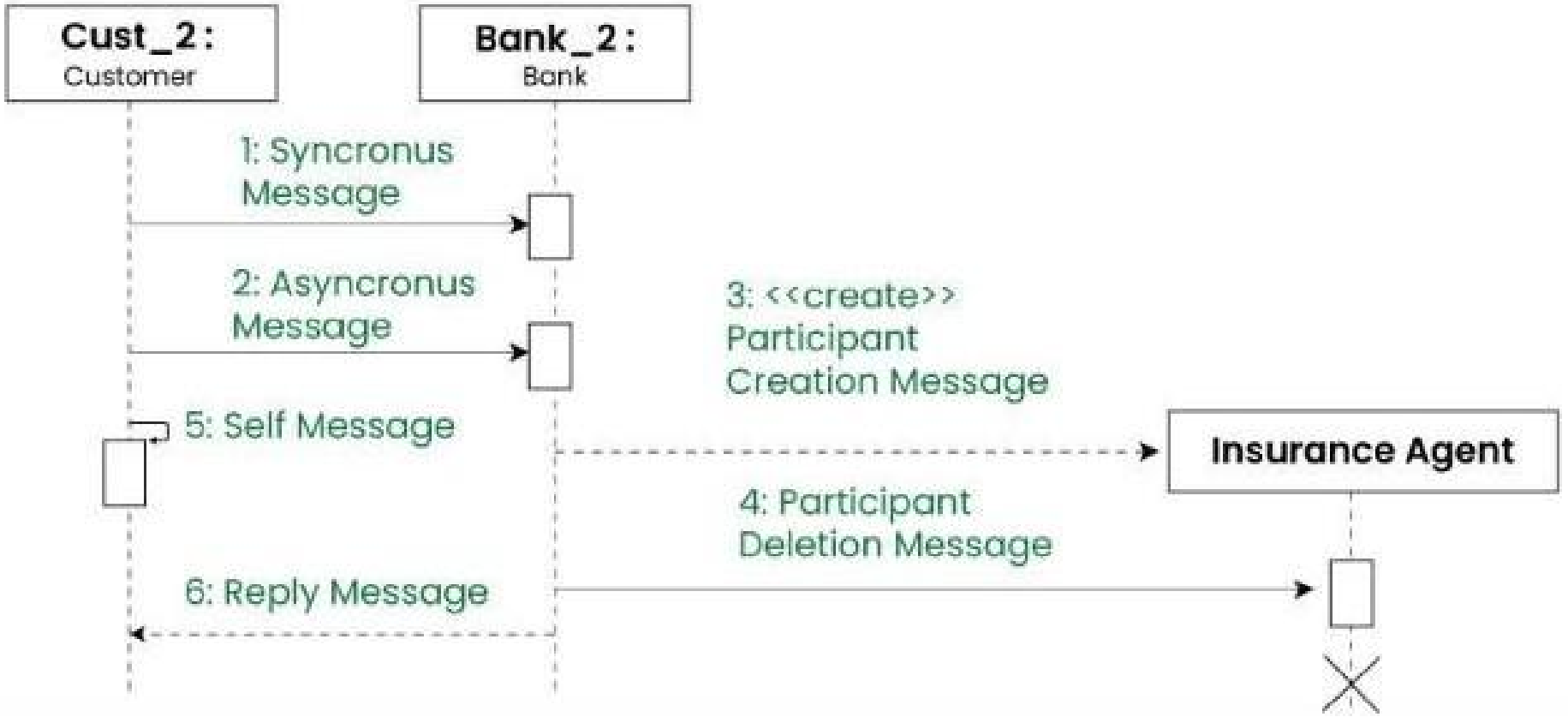
- A lifeline is a named element which depicts an individual participant in a sequence diagram. So basically each instance in a sequence diagram is represented by a lifeline. Lifeline elements are located at the top in a sequence diagram.The standard in UML for naming a lifeline follows the following format:
- Instance Name : Class Name



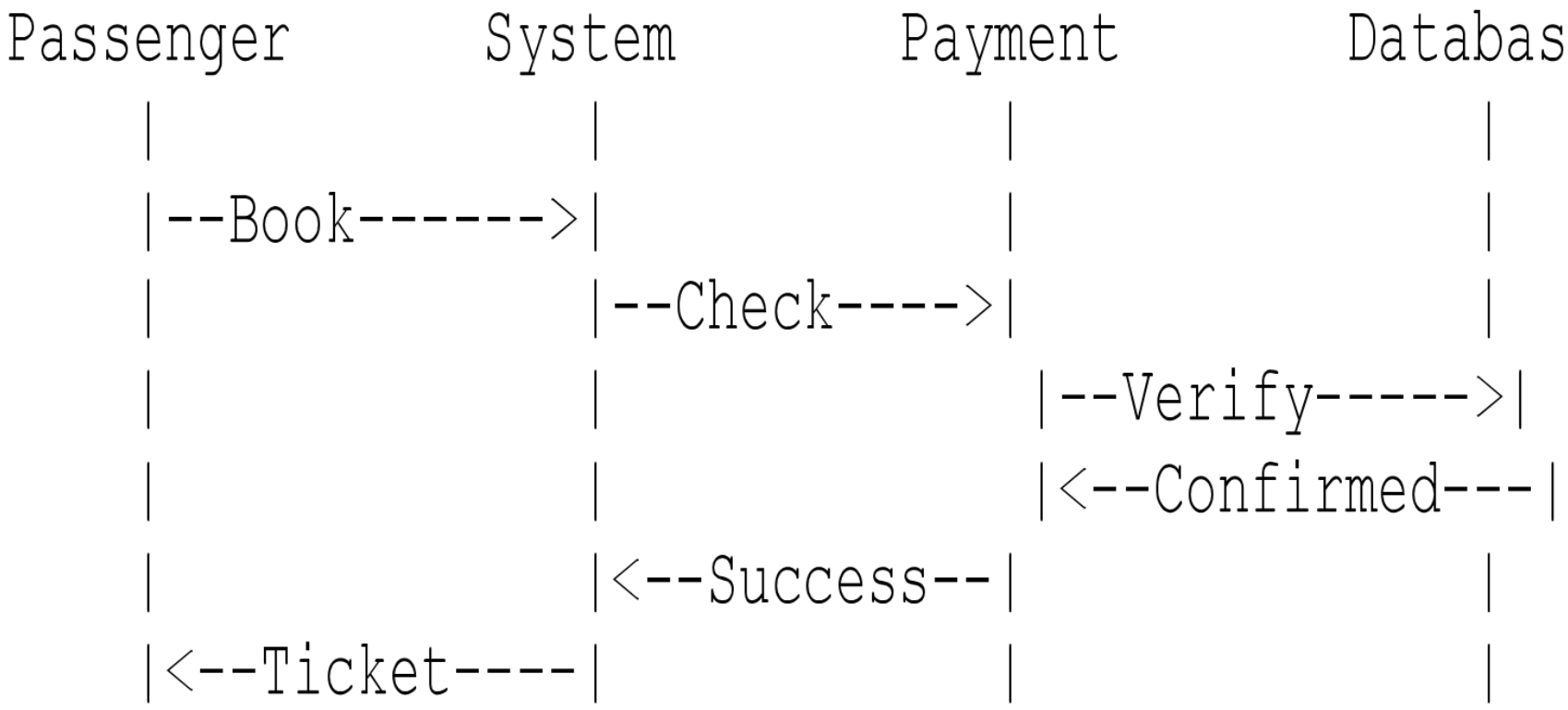
3. Messages

- Communication between objects is depicted using messages.The messages appear in a sequential order on the lifeline.
- We represent messages using arrows.
- Lifelines and messages form the core of a sequence diagram.

Different Types of Messages



Example: booking a bus ticket.



Sequence diagram emphasizes:

- Objects
- Messages
- Time sequence
- Interaction between objects

Activity Diagrams - Unified Modeling Language (UML)

- Activity diagrams are an essential part of the Unified Modeling Language (UML) that help visualize workflows, processes, or activities within a system.
- They depict how different actions are connected and how a system moves from one state to another. By offering a clear picture of both simple and complex workflows, activity diagrams make it easier for developers and stakeholders to understand how various elements interact in a system.
- An Activity Diagram represents a workflow or business process.

Purpose of Activity Modeling

Activity modeling is used to:

- Describe business workflows.
- Show the sequence of activities.
- Model complex processes.
- Identify decision points.
- Represent parallel activities.
- Analyze use cases.
- Improve understanding of system behavior.
- Identify inefficient or unnecessary activities.
- Provide a basis for implementation and testing.

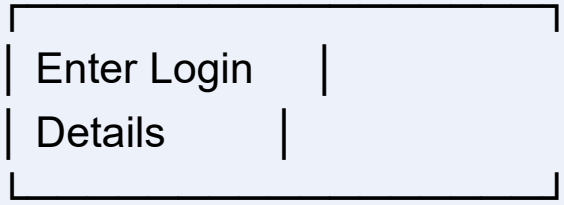
Main Elements of an Activity Diagram

1 Initial Node

Represents the starting point of a process.
It is represented by a filled circle.

2 Activity/Action

Represents an activity performed by the system or an actor.
Example:



3 Control Flow

An arrow shows the order in which activities occur.

Login

|

v

Validate Account

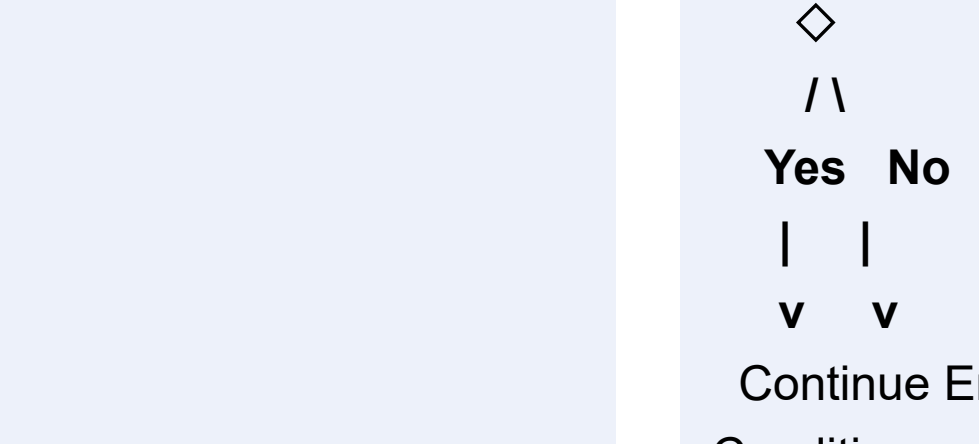
|

v

Display Dashboard

4 Decision Node

A decision represents a condition that determines which path the process follows.



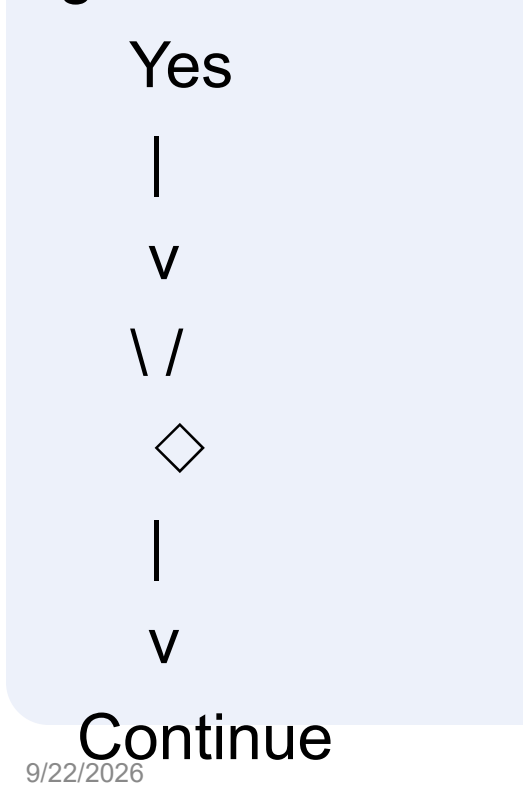
Conditions are usually written in guard brackets:

[valid]

[invalid]

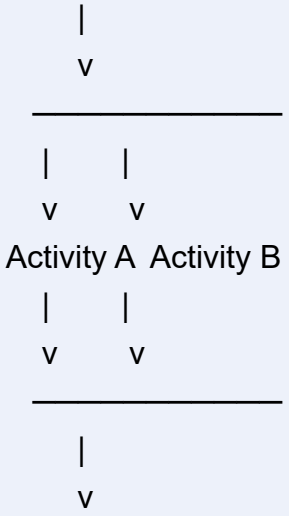
5 Merge Node

A merge brings alternative paths back together.



6 Fork

A fork divides one flow into multiple parallel flows.

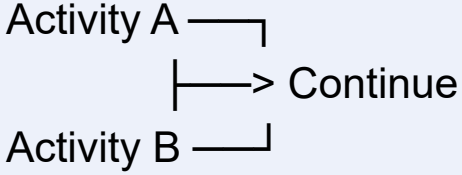


For example, after a customer places an order, the system might simultaneously:

- Send an email.
- Update inventory.
- Prepare an invoice.

7 Join

A join brings parallel activities together.

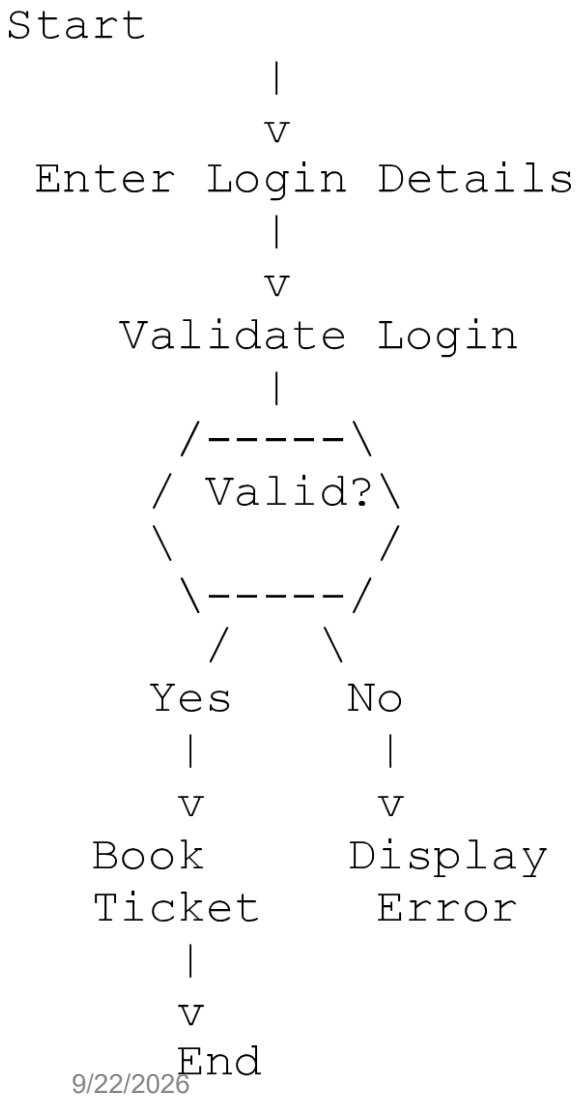


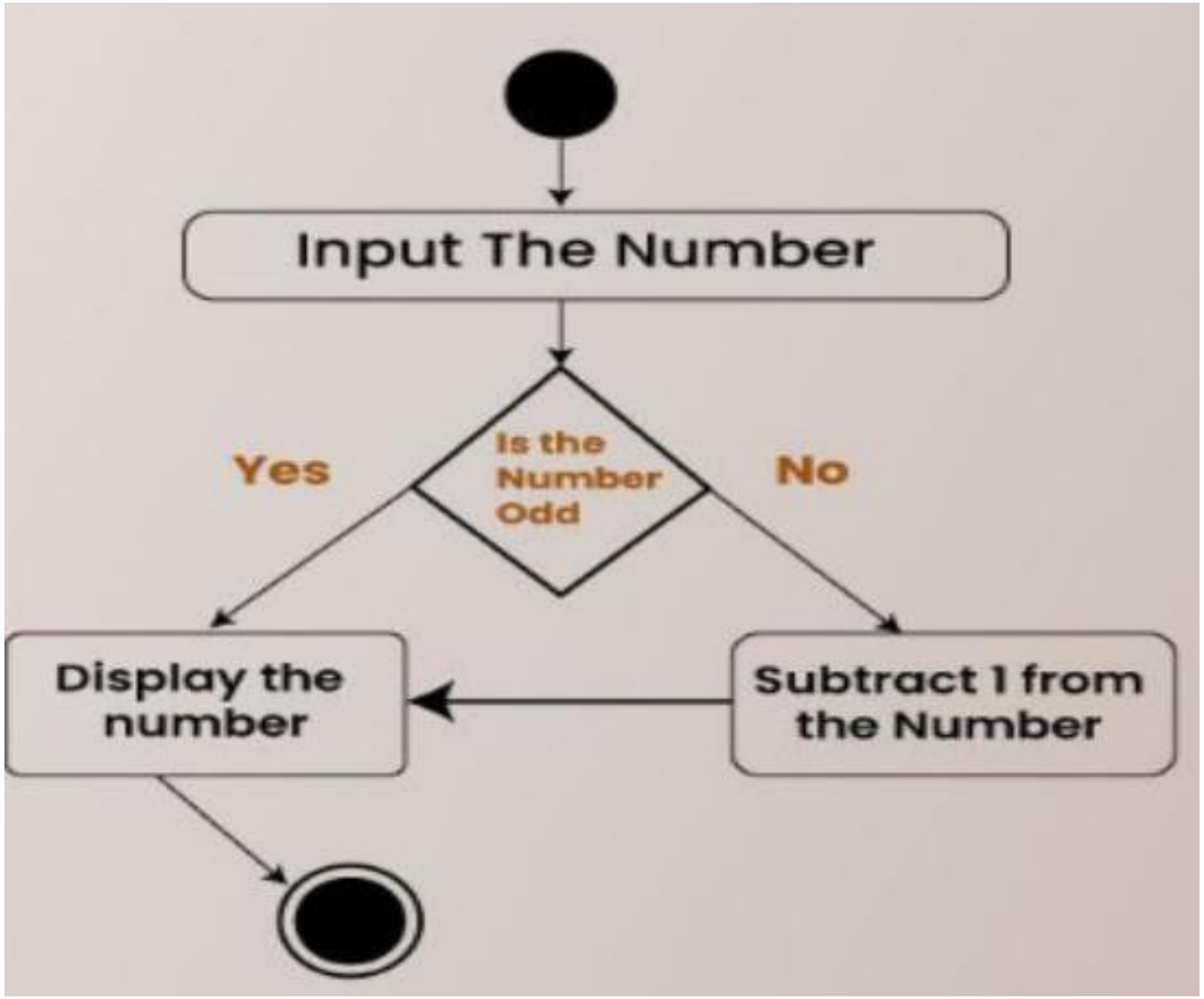
8 Final Node

Represents the end of the activity.

It is commonly represented as:

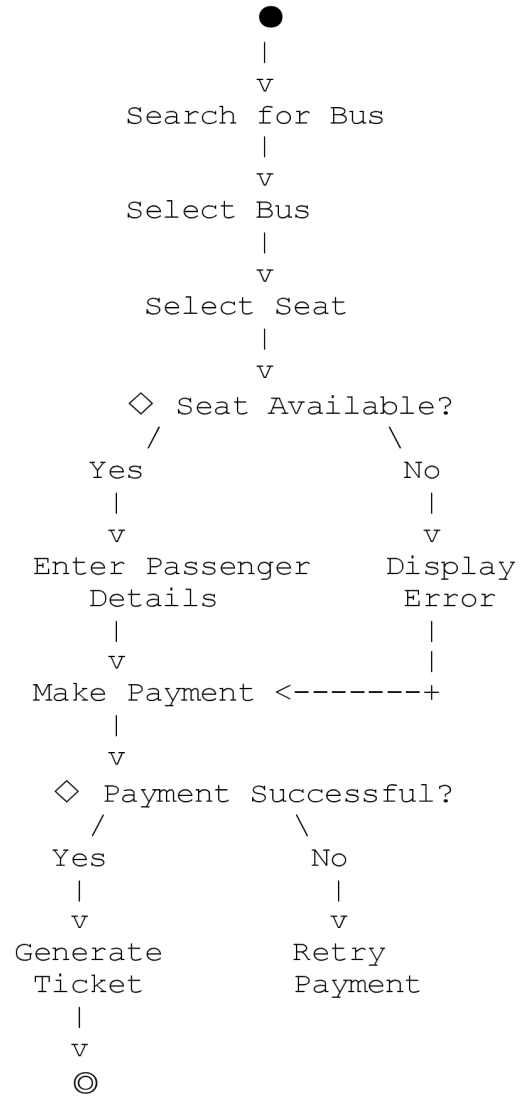
Example


Start
Enter Login Details
Validate Login
Valid?
Yes
No
Book Ticket
Display Error
End


Input The Number
Is the Number Odd
Yes
No
Display the number
Subtract 1 from the Number

Example: Online Bus Ticket Booking Activity

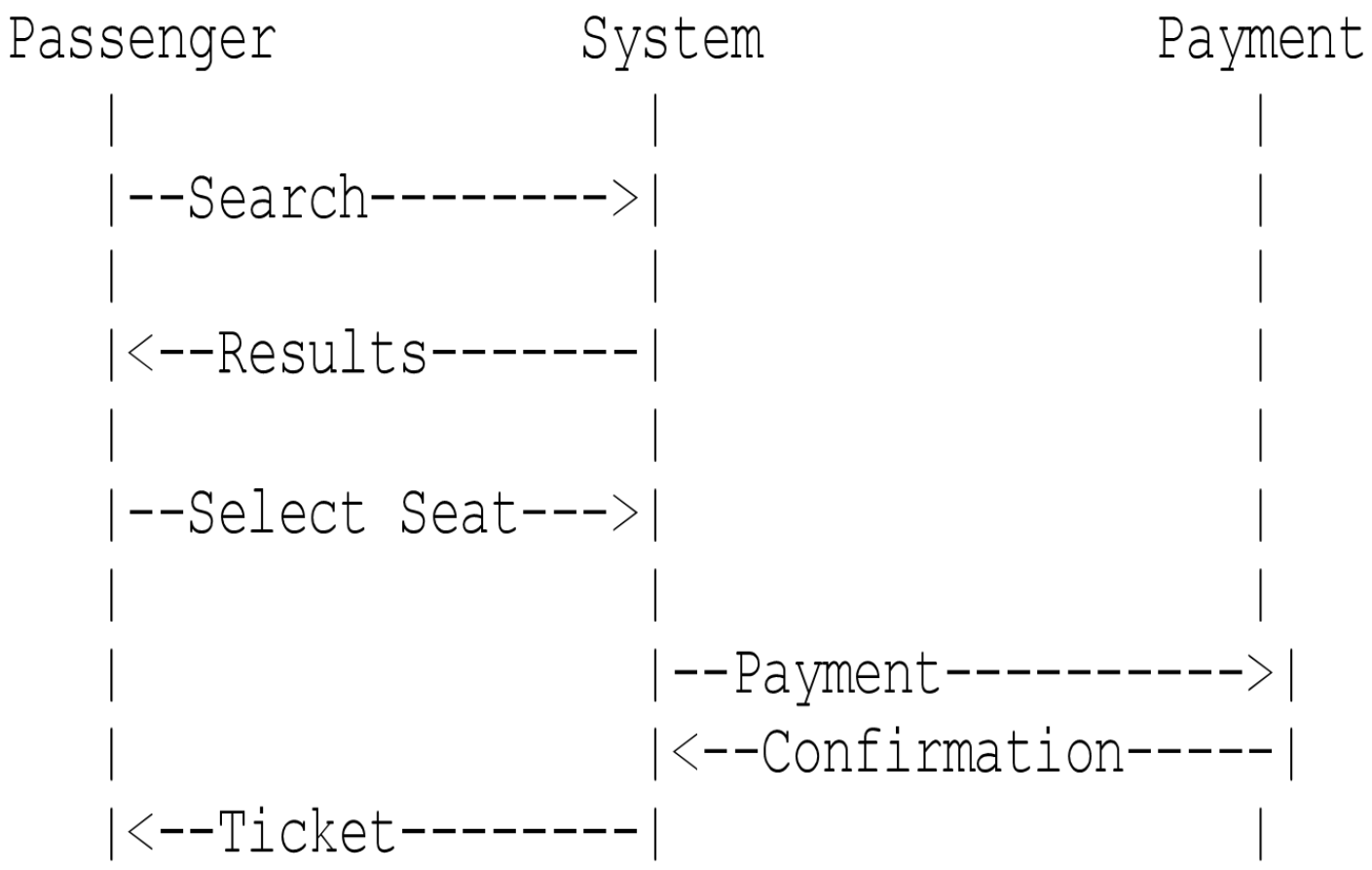
Consider the process of booking a bus ticket.



Activity Partitions / Swimlanes

Activity diagrams can use swimlanes to show which actor or system component is responsible for each activity.

Example



This makes responsibilities clear.

State Modeling

State Modeling describes the different states of an object and how the object changes from one state to another in response to events or conditions.

It is commonly represented using a UML State Machine Diagram.

A state represents a condition or situation in which an object exists for a period of time.

For example, a bus ticket might have the following states:

Available → Reserved → Paid → Confirmed → Cancelled

Purpose of State Modeling

State modeling helps to:

- Understand the life cycle of an object.
- Identify possible states.
- Identify events that cause state changes.
- Define valid and invalid transitions.
- Model event-driven systems.
- Understand how an object behaves over time.
- Identify system responses to events.

State modeling is particularly useful for objects whose behavior changes significantly depending on their current state.

Main Elements of a State Machine Diagram

1 Initial State

Represents where the object's life cycle begins.



2 State

Represents a condition of an object.

Example:



Main Elements of a State Machine Diagram

3 Transition

A transition represents movement from one state to another.

Available ——————————> Reserved

book()

The event causing the transition may be written on the arrow.

4 Event

An event causes an object to change state.

Examples:

bookTicket()

makePayment()

cancelBooking()

expire()

Main Elements of a State Machine Diagram

5 Guard Condition

A guard determines whether a transition can occur.

Example:

Reserved —— pay() [payment valid] ——> Confirmed

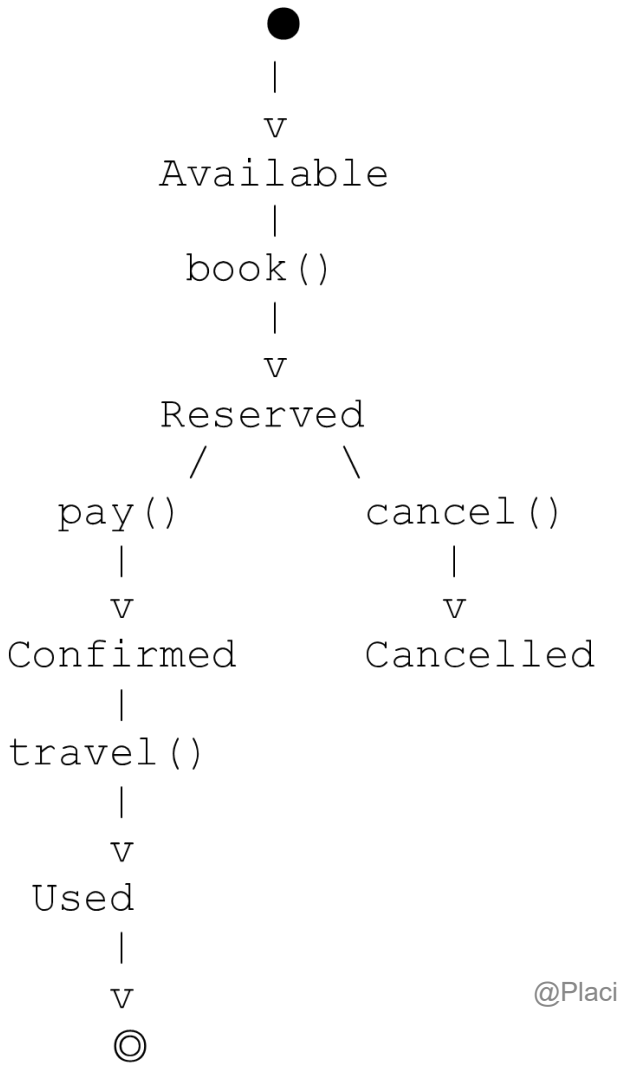
6 Final State

Represents the end of the object's life cycle.

◎

Example: Ticket State Model

An online bus ticket can have several states:



Explanation

1. Initially, the ticket is **Available**.
2. When a passenger books it, it becomes **Reserved**.
3. After successful payment, it becomes **Confirmed**.
4. After the passenger uses the ticket, it becomes **Used**.
5. A reservation can also become **Cancelled**.

Activity Modeling vs State Modeling

Feature	Activity Modeling	State Modeling
Main focus	Workflow/process	Object life cycle
Represents	Activities and actions	States and transitions
Main question	"What happens next?"	"What state is the object in?"
Trigger	Control flow	Events
Best for	Business processes and workflows	Event-driven object behavior
Main diagram	Activity Diagram	State Machine Diagram
Example	Ticket booking process	Ticket status
Emphasis	Flow of work	Change of state

Simple example

Activity modeling:

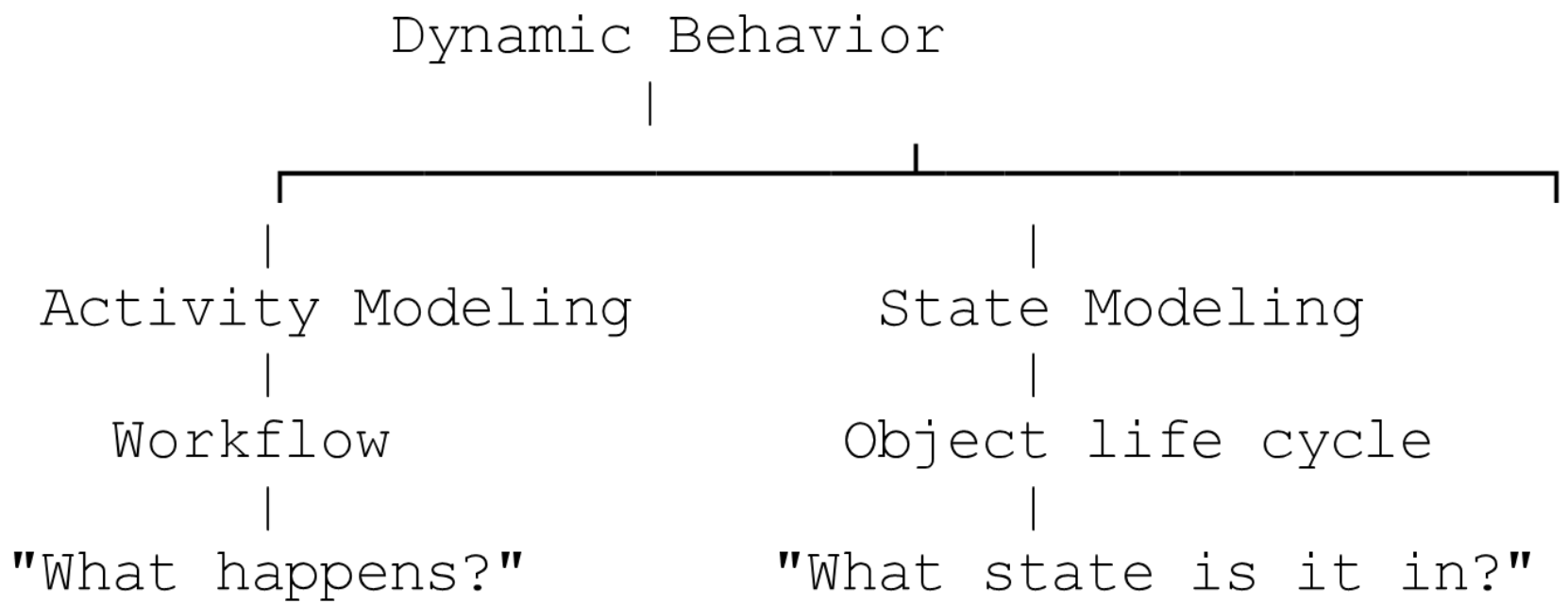
Search bus → Select seat → Pay → Generate ticket

State modeling:

Available → Reserved → Confirmed → Used

Relationship Between Activity and State Modeling

Both models describe **dynamic behavior**, but from different perspectives.



They can be used together.

For an online bus ticket system

Activity model

Shows how a passenger completes a booking:

- **Search → Select → Book → Pay → Receive Ticket**

State model

Shows how the ticket changes:

- **Available → Reserved → Confirmed → Used**

Activity Diagram vs Flowchart

Activity diagrams and flowcharts look similar, but they have different purposes.

Activity Diagram	Flowchart
UML-based modeling technique	General process representation
Designed for software/system modeling	General-purpose process representation
Supports object-oriented analysis and design	Often used for algorithms and procedures
Supports swimlanes	May or may not support responsibility partitions
Supports parallel activities	Traditionally emphasizes sequential flow

When to Use Activity Modeling

Use activity diagrams when you need to model:

- **Business processes**
- **Use case workflows**
- **Approval processes**
- **Registration processes**
- **Transaction processing**
- **Algorithms**
- **Parallel operations**
- **User interactions**

When to Use State Modeling

Use state diagrams when an object:

- **Has several distinct states.**
- **Changes state based on events.**
- **Behaves differently in different states.**
- **Has a clearly defined life cycle.**

Examples include:

Bank account

- **New → Active → Suspended → Closed**

When to Use State Modeling

Order

Created → Paid → Shipped → Delivered

Student application

Submitted → Under Review → Approved → Enrolled

Support ticket

Open → Assigned → In Progress → Resolved → Closed

Steps for Creating an Activity Diagram

1. Identify the process to be modeled.
2. Identify the starting point.
3. List the activities.
4. Arrange activities in logical order.
5. Identify decision points.
6. Identify alternative paths.
7. Identify parallel activities where necessary.
8. Add swimlanes if responsibilities need to be shown.
9. Identify the end point.
10. Review the diagram against the requirements.

Steps for Creating a State Diagram

1. Identify the object to model.
2. Identify its initial state.
3. Identify all important states.
4. Identify events that cause state changes.
5. Define transitions between states.
6. Add guard conditions where necessary.
7. Identify the final state.
8. Check that all meaningful events and transitions are represented.
9. Validate the model against the requirements.

Advantages of Activity Modeling

- Clearly represents workflows.
- Easy to understand.
- Identifies decision points.
- Supports parallel processing.
- Helps identify process problems.
- Useful for business process analysis.
- Supports use-case analysis.
- Helps developers understand system behavior.

Advantages of State Modeling

- Clearly represents an object's life cycle.
- Helps identify valid state transitions.
- Makes event-driven behavior easier to understand.
- Helps identify invalid transitions.
- Useful for designing reactive systems.
- Supports testing of different states and transitions.
- Helps developers implement state-dependent behavior.

Relationship to Other UML Models

Activity and state models complement the other UML diagrams studied earlier:

```
Use Case Modeling
        ↓
"What does the user want?"
        ↓
Class Modeling
        ↓
"What classes are needed?"
        ↓
Sequence/Communication Modeling
        ↓
"How do objects communicate?"
        ↓
Activity Modeling
        ↓
"How does the process/workflow operate?"
        ↓
State Modeling
        ↓
"How do objects change state?"
```

Summary

- Activity Modeling and State Modeling are important techniques for describing the dynamic behavior of software systems.

Activity Modeling

Focuses on **processes and workflows**:

- **Start → Activities → Decisions → Parallel activities → End**

- **State Modeling**
- Focuses on the **life cycle of an object**:

  State → Event → Transition → New State

For example, in a bus ticket system:

Activity Model:

- **Search Bus → Select Seat → Make Payment → Generate Ticket**

State Model:

- **Available → Reserved → Confirmed → Used**

Therefore, activity diagrams explain how work flows through a system, while state machine diagrams explain how an object's condition changes in response to events.

Deployment Diagram in Unified Modeling Language(UML)

- A Deployment Diagram is a type of Structural UML Diagram that shows the physical deployment of software components on hardware nodes. It illustrates the mapping of software components onto the physical resources of a system, such as servers, processors, storage devices, and network infrastructure.
- They show where software components are placed on hardware devices and shows how they connect with each other.This diagram helps visualize how the software will operate across different devices

Components and Notations in Deployment Diagram

- **Nodes:** These represent the physical hardware entities where software components are deployed, such as servers, workstations, routers, etc.
- **Components:** Represent software modules or artifacts that are deployed onto nodes, including executable files, libraries, databases, and configuration files.
- **Artifacts:** Physical files that are placed on nodes represent the actual implementation of software components. These can include executable files, scripts, databases, and more.



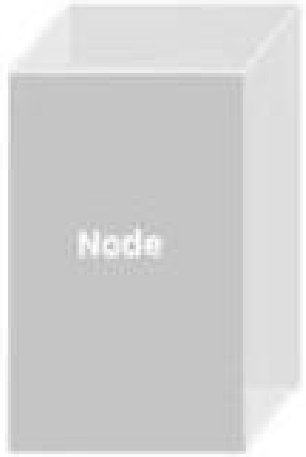
COMPONENT



ARTIFACT



INTERFACE



NODE

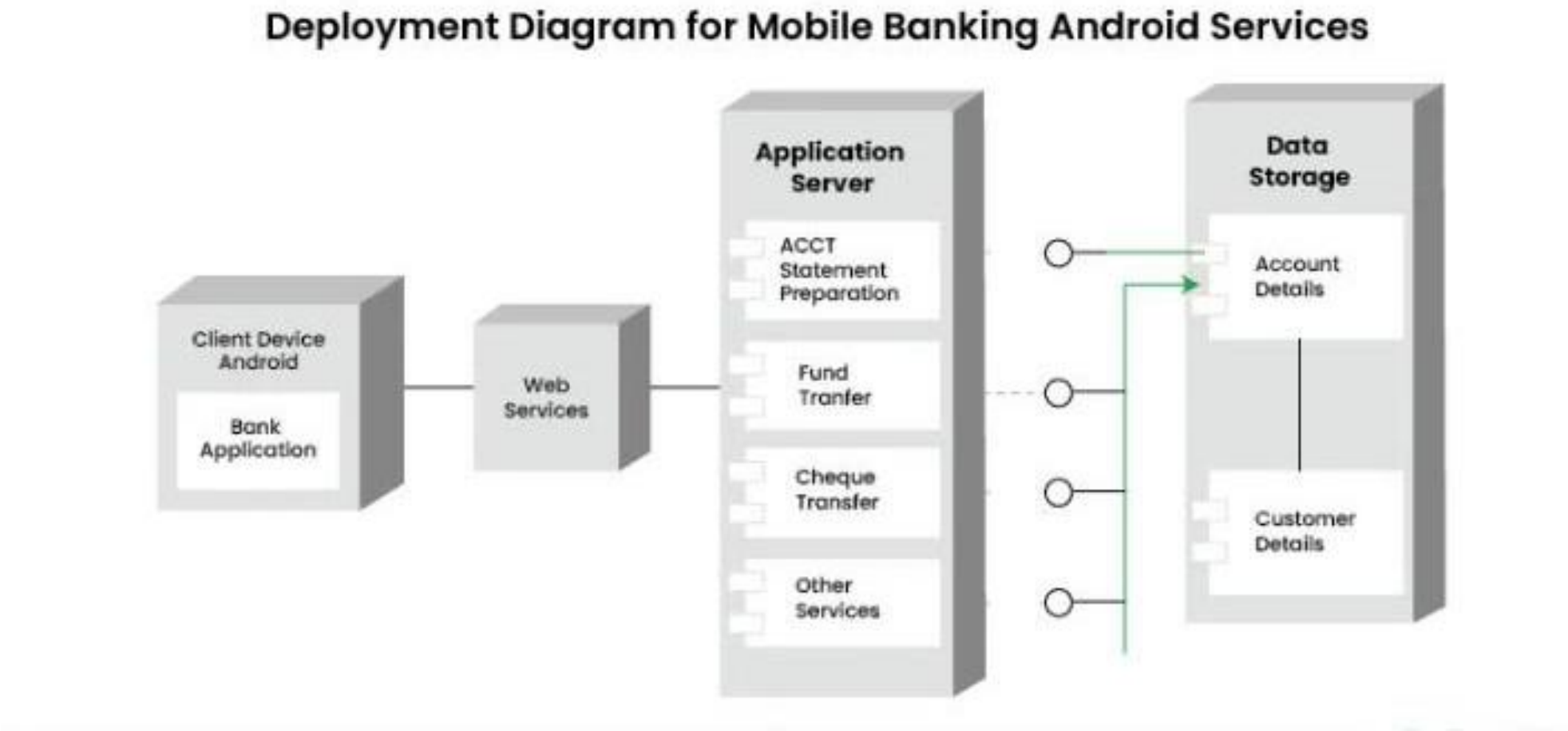
- A **component represents** a modular and reusable part of a system, typically implemented as a software module, class, or package. It encapsulates its behavior and data and can be deployed independently.
- **An artifact represents** a physical piece of information or data that is used or produced in the software development process. It can include source code files, executables, documents, libraries, configuration files, or any other item.
- **An interface** defines a contract specifying the methods or operations that a component must implement. It represents a point of interaction between different components or subsystems.
- **A node represents** a physical or computational resource, such as a hardware device, server, workstation, or computing resource, on which software components can be deployed or executed.

Steps for creating a Deployment Diagram

- Step1: Identify Components: List all software parts and hardware devices that will be in the deployment diagram.
- Step 2: Understand Relationships: Figure out how these parts connect and work together.
- Step 3: Gather Requirements: Collect details about hardware, network setups, and any special rules for deployment.
- Step 4: Draw Nodes and Components: Start by drawing the hardware devices (nodes) and software parts (components) using standard symbols roughly at first improvise it and draw the final one.

- Step 5: Connect Nodes and Components: Use lines or arrows to show how nodes and components are linked.
- Step 6: Add Details: Label everything clearly and include any extra info, like hardware specs or communication protocols.
- Step 7: Documentation:Write down any important decisions or assumptions made while creating the diagram.

Deployement Diagram For Mobile Banking Andorid Services Example 1



Assignment No 2 of Software Modeling Design

For the same software project topic develop the following design for both Structure and behovior Design :

1. Use case diagram
2. Sequence diagram
3. Class diagram
4. Activity diagram (Swimlane)
5. State Diagram

Note : Use the UML software to design all diagrams and printed work is acceptable

Deadline is fixed to next class time

